

GBIN6U03 - Projet Logiciel

Guillaume Huard

Université Grenoble Alpes

Rappel concernant l'IA

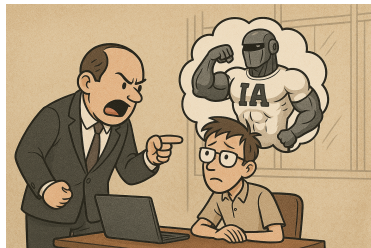
Le travail universitaire n'est pas fait pour faire monter en compétence l'IA, mais vous !



Et au travail, l'IA ne compensera pas votre manque de recul



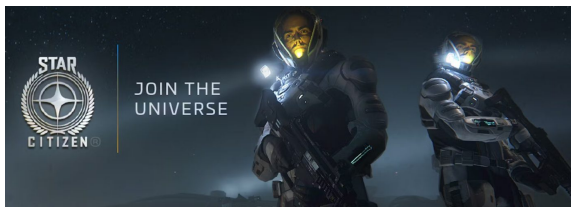
En examen, seules vos propres compétences comptent



Objectif

Réalisation d'un logiciel de jeu

- 1 mois à temps complet
- Réinvestissement des acquis
- Travail en équipe (6 étudiants)
- Autonomie (gestion de projet)
- Taille moyenne (>6 HM)



Début du développement en 2012, coût $>$ \$750M

Un jeu

Motivant, Cohérent par rapport à la formation

- On retrouve de l'algo (IA) dans les jeux proposés
- Possibilité de jeu en réseau (Système)
- La programmation de l'ensemble requiert un effort de design/spécification, test, intégration, suivi
- Occasion de découvrir l'IHM

Niveau adapté

Forme

Jeux judicieusement choisis

- Qui amènent à travailler sur les thèmes des UE
- Dont la difficulté de réalisation est du même ordre

Peu de contraintes

- En java ≥ 5 et ≤ 11 , avec toute sa bibliothèque standard
- Indépendant de la résolution
- Pas de squelette donné (réutilisation du Sokoban autorisée)

Support limité

- Support enseignant dans le périmètre des UEs

Déroulement

Contenu à préciser au fur et à mesure sur la page du projet

Semaine	Contenu
1	- démarrage du préprojet
2	- amphis : jeux, IHM, threads, réseau, IA (Minmax) - présentation des préprojets réalisés
≥ 3	Détails à préciser : - audits : $IHM \times 2$, $IA \times 2$, code - amphi : soutenance - soutenances

Support additionnel

- Tuteurs (1 par groupe) une fois le jeu choisi (semaine 2)
- Hotline : `im2ag-l3-info-hotline-s6 AT univ-grenoble-alpes.fr`
- Forum : : sur `im2ag-tchat.univ-grenoble-alpes.fr` dans le canal PROG6 de l'équipe L3 INF 24-25

Cahier des charges

Votre logiciel final DOIT proposer les fonctionnalités suivantes

- Jeu à deux joueurs
 - humain contre humain
 - humain contre IA
 - choix du joueur qui commence
 - coup choisi aléatoirement parmi les meilleurs pour l'IA
- Historique : annuler / refaire sans limite (on doit pouvoir annuler tous les coups jusqu'à la position initiale)
- Sauvegarder / charger avec historique complet
- Nouvelle partie (abandonner la partie en cours)
- Pas de musique, ou musique désactivée par défaut

Vous êtes libre d'y ajouter autant d'autres fonctionnalités supplémentaires que vous le souhaitez

A rendre le 28/05/2025 à 14h en F108

Code source

- Sur le dépôt Rendus-prog6-2024-2025/groupe-X sur <https://gricad-gitlab.univ-grenoble-alpes.fr> les droits vous seront donnés une fois les groupes constitués

Binaires fonctionnels

- Archive exécutable (jar) à rendre sur clé USB
 - contenant TOUT le jeu (y compris images, sons, etc. pas de ressources extérieures)
 - ne contenant PAS les sources ni le dépôt git

Votre jeu sera évalué sur la base des binaires rendus, mais

- bugfix possible pour la version que vous montrez en soutenance
- version différente possible pour la soutenance
 - pour montrer des information cachées (état du jeu, coups retenus par l'IA, ...)
 - pour montrer les évolutions de l'IHM
 - ...

A rendre avant de démarrer la soutenance

Dossier de validation IA (2 exemplaires papier + pdf sur dépôt)

- Description de l'algorithme
 - typiquement, détail de l'heuristique si minmax
 - justification (voire preuve) de vos choix
- Synthèse des évaluations effectuées
 - niveaux de difficulté bien distincts et bien ordonnés
 - validation de chaque critère pris en compte par l'heuristique
 - jeu contre des humains

Dossier de validation IHM (2 exemplaires papier + pdf sur dépôt)

- Description l'interface
 - éléments importants du design
 - justification intuitive des solutions retenues
- Synthèse des évaluations effectuées
 - validation des différents éléments d'interface
 - clarté de l'interface (lisibilité, feedforward)
 - ergonomie (utilisation aisée, feedback approprié)
 - adéquation au public ciblé

Evaluation

1/4 Présentation orale

- Clarté, respect du temps, réponse aux questions

1/4 IHM

- Qualité, progression au fil des audits, validation

1/4 IA

- Qualité, validation

1/4 Qualité technique globale

- Robustesse, fonctionnalités

Le préprojet

Gaufre empoisonnée

Objectifs

- Tester le projet sur une petite échelle
- Réfléchir à l'architecture logicielle de votre jeu
- Appréhender le travail au sein de votre groupe

Mise en œuvre

- 3 jours de travail
- Revue de la part des enseignants le 23 avril (en présentiel)

Non noté



Démo

Contraintes

Même cahier des charges que pour le jeu final

Quelques précisions

- IHM
 - Claire : état du jeu, tour, score, fonctionnalités
 - Ergonomique : organisation, utilisation
- Niveaux pour l'IA
 - Aléatoire
 - coup gagnant/perdant
 - Arbre et/ou

Conseils d'organisation

Conduite du développement

Identification des besoins

- Triés par priorité
- Temps de développement estimé

Architecture globale possible (3 équipes)

- Arbitre (Moteur de jeu)
- Interface (IHM)
- Joueur (IA)

Spécification

- Relation entre les modules
- Interfaces

Cycle de développement

Privilégier les cycles courts

- Spécification
- Implémentation
- Test

Implémenter plus que nécessaire

- Code de test plus volumineux que le code testé
- Outils, code temporaire
 - Editeur de plateaux de jeu
 - Joueur qui joue une séquence fixe pour tester l'Arbitre
 - Traces d'exécution

Rôles

Correspondent à des tâches globales

- Suivre l'avancement du projet
- Valider chacune des parties au fur et à mesure
- Intégrer l'ensemble des contributions

Typiquement un responsable mais

- peut impliquer plusieurs personnes
- le responsable délègue et collecte les résultats

Constituent les fondations du projet

- Projet fonctionnel
- Réalisation dans les temps

Le rôle de rapporteur

Tenir un journal

- Pistes envisagées / essayées
- Avancée dans les tâches
- Respect du planning

Communiquer régulièrement

- Au niveau de l'ensemble du groupe
- Avec le tuteur

Susciter des réunions pour réagir selon l'avancement

- changer la priorité des tâches
- annuler les tâches superflues
- réaffecter les forces aux tâches prioritaires



Le rôle de testeur

Tester le code à plusieurs niveaux

- Tests unitaires au sein d'un module
- Tests d'intégration entre modules
- Test de déploiement du logiciel sous sa forme finale
- Tests de réponse aux besoins

Conserver et automatiser les tests

- Non régression
- Identification plus aisée des bugs

Tester et faire tester régulièrement



Le rôle d'intégrateur

Intégrer à intervalles réguliers

- Pas de divergence par rapport aux spécifications
- Pas de bug d'intégration



Conserver les versions successives du logiciel

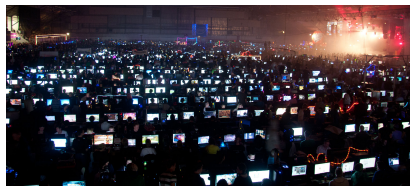
- En étiquetant ("tag") les versions stables
- Permet de facilement revenir en arrière en cas de fausse piste
- Versions stables utilisées pour les évaluations

Evaluations auprès d'utilisateurs

Définir le public visé : âge, activité, expertise, ...

Programmer des évaluations régulières

- Sur les versions successives de votre logiciel
- En conservant inchangée une partie du public



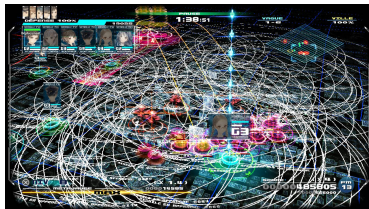
Garder une trace écrite de toute évaluation

- Public testé
- Résultat (votre avis et celui du sujet testé)
- Retour sur l'identification des besoins

L'évaluation de l'IHM

Clarté

- Compréhension des fonctionnalités
- Identification des possibilités



Ergonomie

- Facilité de navigation
- Scénario d'usage typique favorisé

Feedback

- Retours visuels en cas d'action de l'utilisateur
- Notifications en cas de changement d'état de l'application

Réponse aux besoins

L'évaluation de l'IA

Bien fondé de la stratégie

- Heuristiques argumentées
- Validation de chaque heuristique

Bien fondé des niveaux de difficulté

- L'IA difficile bat la moyenne qui bat la facile
- Tester les deux scénarii pour les jeux asymétriques

Comportement contre l'humain

- Adéquation difficulté / expertise du sujet testé
- Pas de comportement pathologique

Temps de calcul approprié



QUESTIONS

